



LECTURE 12 — SPACE COMPLEXITY & SELECTION SORT

DSA ka asli matlab: sochna seekho, sirf yaad mat karo



CHAPTER VISION — WHY THIS MATTERS

Is chapter ka goal sirf definitions ya algorithms ratna nahi hai.

Asli aim ye clarity banana hai ki:

- Memory kab, kyu aur kitni use ho rahi hai
- Koi bhi new problem aaye to approach kaise karein
- Sorting jaise basic lagne wale problems ke peeche ka real-life logic

👉 Agar ye chapter samajh gaye,
to **DSA tumhare liye subject nahi, ek core skill ban jayega.**



SPACE COMPLEXITY — MEMORY KI REALITY

Space Complexity kya hoti hai?

Space Complexity batati hai:

Program ko run karne ke liye
kitni memory (RAM) chahiye,
input size ke respect me.

⚠ Important baat:
Sirf variables hi count nahi hote, balki:

- Arrays
- Recursion stack
- Temporary data structures

👉 **Sab space complexity ka part hote hain.**

SPACE COMPLEXITY KE TYPES (DSA ME BAS 2)

1 Input Space

Input data ko store karne ke liye jo memory lagti hai.

```
int arr[n];
```

 Ye memory **unavoidable** hoti hai, isliye **analysis me usually ignore** kar dete hain.

2 Auxiliary Space (MOST IMPORTANT)

Problem solve karte waqt jo **extra space** use hoti hai.

 **Interview aur real systems dono me isi ko judge kiya jaata hai.**

 **Formula**

```
Total Space = Input Space + Auxiliary Space
```

Example 1 — Simple Loop (NO EXTRA SPACE)

```
for(int i = 0; i < n; i++) {  
    cout << i;  
}
```

Deep Understanding

- `i` → sirf ek variable → constant memory
- Koi new array, recursion ya dynamic allocation nahi

 **Auxiliary Space = $O(1)$**

 **Space Complexity = $O(1)$**

Golden Line

Loop kitni bhi baar chale,
agar nayi memory allocate nahi ho rahi → **space hamesha constant rahegi.**

Example 2 — Reverse Array using New Array

```
int arr[n] = {10, 20, 30, 40};  
  
int newArr[n];
```

```
int j = n - 1;  
  
for(int i = 0; i < n; i++) {  
    newArr[i] = arr[j];  
    j--;  
}
```

```
return newArr;
```

Space Breakdown (Crystal Clear)

- `arr[n]` → Input Space
- `newArr[n]` → Auxiliary Space ($O(n)$)
- `i` , `j` → Constant Space ($O(1)$)

 Auxiliary Space = $O(n)$

 Total Space Complexity = $O(n)$

 Rule yaad rakho

Jahan `n-size` ka naya structure (array / vector / map) bana →
space = $O(n)$ pakka.

INTERVIEW THUMB RULES (MUST REMEMBER)

-  New array / vector / map → $O(n)$
 -  Recursion → stack space automatically add hoti hai
 -  Sirf variables → $O(1)$
-

🤔 Space Complexity important kyu hai?

- 📱 Mobile apps → RAM limited hoti hai
- ⚙️ Embedded systems → memory bahut kam hoti hai
- 🏠 Large-scale products → memory = direct cost

👉 Kam space use karne wala code zyada scalable hota hai.

🧠 DSA KYU PADHATE HAIN? (REALITY CHECK)

- ❌ IIT me padhna = genius hona (MYTH)
- ❌ 1000 DSA questions = best engineer (MYTH)

✅ Reality

- IIT → opportunities zyada
- Skill → **khud build karni padti hai**

📌 Companies DSA isliye puchti hain kyunki:

- Tum **nayi problem ko kaise todte ho**
- Tum **kaise logically sochte ho**
- Tum **trade-offs aur constraints** samajhte ho

💡 DSA = Thinking Framework

💡 Dev / AI / System Design = Us thinking ka application

📊 SORTING — REAL LIFE SE DIRECT LINK

📌 Problem Statement

Ek array di gayi hai, use **sort** karna hai.

```
arr = {5, 3, 8, 1, 2}
```

Sorting ke Real-Life Uses

- 🛒 Amazon → Price Low → High
- 🏫 College → Height-wise student line
- 📊 Analytics → Ranking & reports
- 🤖 AI → Data cleaning & preprocessing

👉 Sorting ke bina system **slow aur inefficient** ho jaata hai.

Common Sense Approach (College Analogy)

Socho students line me khade hain:

- Sabse chhota student dhundo
- Use line ke aage laga do
- Bache hue students me fir se sabse chhota dhundo
- Ye process repeat karo



👉 Ye natural process hi Selection Sort hai.

SELECTION SORT — FIRST PRINCIPLE SE

Core Logic

- Har step me **minimum element** find karo
 - Use uski **correct position** par place karo
 - Har pass ke baad **1 element permanently sorted** ho jaata hai
-

Selection Sort Code (Clean & Fully Commented)

```
vector<int> selectionSort(vector<int>& arr) {  
    int n = arr.size();
```

```
for(int i = 0; i < n - 1; i++) {  
    int index = i;    // maan liya current element minimum hai  
  
    for(int j = i + 1; j < n; j++) {  
        if(arr[j] < arr[index]) {  
            index = j;    // naya minimum mil gaya  
        }  
    }  
  
    // minimum element ko sahi position par rakh do  
    swap(arr[i], arr[index]);  
}  
  
return arr;  
}
```

Time Complexity — Selection Sort

- **Worst Case** → $O(n^2)$
- **Best Case** → $O(n^2)$

 Reason:

Selection sort **har case me poora array scan** karta hai, chahe array pehle se sorted hi kyu na ho.

Space Complexity — Selection Sort

- Sirf kuch variables (**i**, **j**, **index**)
- Koi extra array ya data structure nahi

- ✓ Auxiliary Space = $O(1)$
 - ✓ In-place sorting algorithm
-

🧠 Selection Sort kab use kare?

- Jab memory limited ho
- Jab input size chhota ho
- Jab logic simple aur predictable chahiye

✗ Large datasets ke liye avoid karo.

🌟 FINAL CORE MESSAGE

- Computer khud fast nahi hota
- Tumhari thinking fast hoti hai

DSA ka goal:

- ✗ Ratna nahi
 - ✓ Sochna seekhna
-

📖 ULTRA QUICK SUMMARY (1-MIN REVISION)

- 🧠 **Space Complexity** → Input ke saath memory usage
- 📁 **Auxiliary Space** → Solution ke dauran use hui extra memory
- ✓ **$O(1)$ Space** → Koi extra data structure nahi
- 📈 **$O(n)$ Space** → New array / recursion stack use hua
- 🔁 **Selection Sort** → Minimum element pick karke correct position par place
- ⌚ **Time Complexity** → $O(n^2)$
- 📄 **Space Complexity (Selection Sort)** → $O(1)$
- 🎯 **DSA** → Problem-solving & logical thinking framework